

Lab Report: Sorting Algorithms

Course: 2XC3 - Algorithms and Data Structures

Prepared By:

Ben Guo
Heeman Ijaz
Jason Xie

Submitted To:

Dr. Ilir Dema
Department of Computing and Software
McMaster University

January 31, 2026

Contents

Executive Summary	1
1 Experiment 1: Run Times of Bad Algorithms	2
1.1 Objective	2
1.2 Methodology	2
1.3 Results	2
2 Experiment 2: Improved Bubble & Selection Sort	3
2.1 Objective	3
2.2 Methodology	3
2.3 Results	4
3 Experiment 3: Bad Algorithms on Special Lists	6
3.1 Objective	6
3.2 Methodology	6
3.3 Results	6
4 Experiment 4: Run Times of Good Algorithms	8
4.1 Objective	8
4.2 Methodology	8
4.3 Results	8
5 Experiment 5: Optimal Conditions for Quick Sort	9
5.1 Objective	9
5.2 Methodology	9
5.3 Results	9
6 Experiment 6: Better Quick Sort	11
6.1 Objective	11
6.2 Methodology	11
6.3 Results	12
7 Experiment 7: Iterative Merge Sort	13
7.1 Objective	13
7.2 Methodology	13
7.3 Results	13
8 Experiment 8: Insertion Sort vs 'Good' Sorts	15
8.1 Objective	15
8.2 Methodology	15
8.3 Results	15
A Code Navigation Guide	18

List of Figures

1	run times of the three bad sorts	2
2	performance of bubble sort vs improved bubble sort	4
3	performance of selection sort vs improved selection sort	5
4	run time of bad sorts on lists with 0 swaps, up to 3301 swaps	6
5	run time of the three good sorts	8
6	performance of three good sorts on reversed lists with x swaps	9
7	performance of quick sort with 1 pivot vs 2 pivots	12
8	iterative merge sort outperforms recursive for large lists	14
9	insertion sort can only compete up to lists of length ≈ 50	16
10	insertion sort performance correlates directly with the number of inversions .	16
11	insertion sort struggles with reverse sorted lists	17

Executive Summary

In this lab, we dove deep into both bad and good sorting algorithms; exploring their specific strengths and weaknesses, as well as optimizations and an analysis of their behavior. Through this, we practiced how to test algorithms and learned the parameters needed to fine tune our experiments and properly express our results. Here are a few of the main topics we explored:

Key Takeaways

- **What makes a sort 'good' or 'bad':** There is the general classification of what a 'bad' sort is and what a 'good' sort is, however each of the respective sorts have their own pros and cons, as well as intermixing that can be better than any single sort alone.
- **Sorting behaviors on special lists:** Not every sort is made equally, some are better on lists that might be close to sorted, some are worse. In real-world scenarios, there are a wide variety of circumstances and as such certain algorithms have their own uses.
- **Making sorting algorithms better:** Of course, there are the versions of algorithms that we learn about in class, a baseline implementation that does exactly what we outline. However, this is usually not the most efficient method, and so we explore the various ways to optimize algorithms by taking advantage of the way they work.
- **Nuances in run time:** The general idea behind an algorithm's run time is its time complexity. However, in many cases, two $O(n \log n)$ or $O(n^2)$ algorithms can have wildly different performances due to large constant factors. For example, if n is small enough, 1000 constant operations might slow down the code more than the base n^2 complexity, even though quadratic performance should be worse than constant performance. We explore this nuance when looking at both good and bad algorithms.
- **Implementation choices:** Sometimes, we need to use recursion to solve a problem. However, we know generally that iteration beats recursion, and so we might need to adapt a simple recursive problem into a more difficult iterative problem to improve performance. In that same vein, we experimented with different trade-offs in these algorithms: The aforementioned recursion vs iteration for merge sort, how we choose pivots for quick sort, how we swap values in bubble sort, etc.

1 Experiment 1: Run Times of Bad Algorithms

1.1 Objective

Our goal is to run trials on the three 'bad' sorting algorithms: bubble sort, insertion sort, and selection sort. We will investigate their behavior on both large and small lists. While they are all considered bad, some of them must be better than others due to the innate differences that cannot simply be expressed with an $O(n^2)$ run time that they share.

1.2 Methodology

For bad sorts, we cannot test with values that are too large as the ballooning complexity will be difficult to run on our machines. Thus, we tested about 20 lists, from a length of 100 to 2000. For each list, we ran 3 tests and took the best run time in order to minimize outside interference or hiccups that might create outliers. Even this small range and number of lists was enough to show a clear trend.

1.3 Results

First, using Figure 1, we can tell all of these sorts have a baseline complexity of $O(n^2)$ due to the large increase for even small n for all algorithms. We can further observe that bubble sort is by far the worst, rocketing past the rest. This is followed by insertion sort, and finally, selection sort, as the 'best' of the bad sorts.

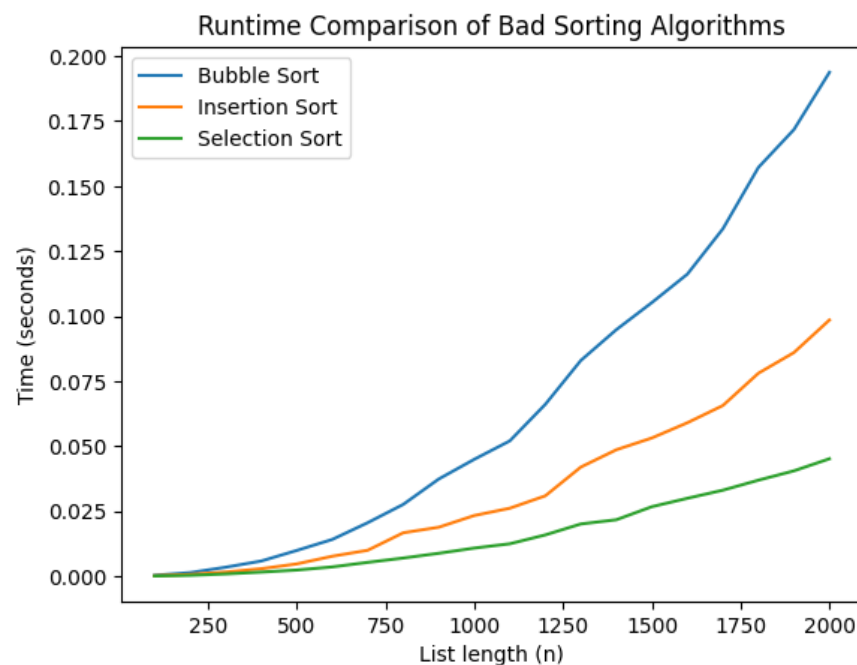


Figure 1: run times of the three bad sorts

2 Experiment 2: Improved Bubble & Selection Sort

2.1 Objective

Our goal is to implement **improved** versions of both bubble and selection sort. After understanding and implementing the two, we will run tests with both the original and the improved versions, noting the differences in run time and what it implies for the overall complexity of both algorithms.

2.2 Methodology

First, the optimization for bubble sort was the same as how we improved insertion sort in lecture: while we still do k comparisons, we reduce the overhead of swapping by only performing 1 insert after $k - 1$ shifts.

Listing 1: Improved bubble sort

```
def bubble_sort2(L):
    n = len(L)
    for i in range(n):
        j = 0
        while j < n - 1 - i:
            if L[j] > L[j + 1]:
                value = L[j]
                k = j + 1

                while k < n - i and L[k] < value:
                    L[k - 1] = L[k]
                    k += 1

                L[k - 1] = value
                j = k
            else:
                j += 1
```

For selection sort, instead of finding just the smallest element on each pass, we find the largest and smallest element and put both of them at the respective ends, cutting the number of iterations for the outer loop roughly from n to $\frac{n}{2}$ (though we increase the work of the inner loop).

Listing 2: Improved selection sort

```
def selection_sort2(L):
    left = 0
    right = len(L) - 1

    while left < right:
```

```

min_index = left
max_index = left
for i in range(left, right + 1):
    if L[i] < L[min_index]:
        min_index = i
    if L[i] > L[max_index]:
        max_index = i
swap(L, left, min_index)
if max_index == left:
    max_index = min_index
swap(L, right, max_index)
left += 1
right -= 1

```

To test the lists, we check a similar range as we did for the three bad sorts previously. We test 20 lists of lengths from 100 - 2000 with 3 trials per list. This way we get to see the behavior of these lists in a manageable time.

2.3 Results

Bubble sort is $O(n^2)$ in complexity, and even our improved version cannot avoid this. However, Figure 2 shows a marked improvement, especially for lists of length up to 2000. It makes sense; we do not inherently change the operation of bubble sort, but we remove some redundancy which adds up when we have to make tens of thousands of swaps.

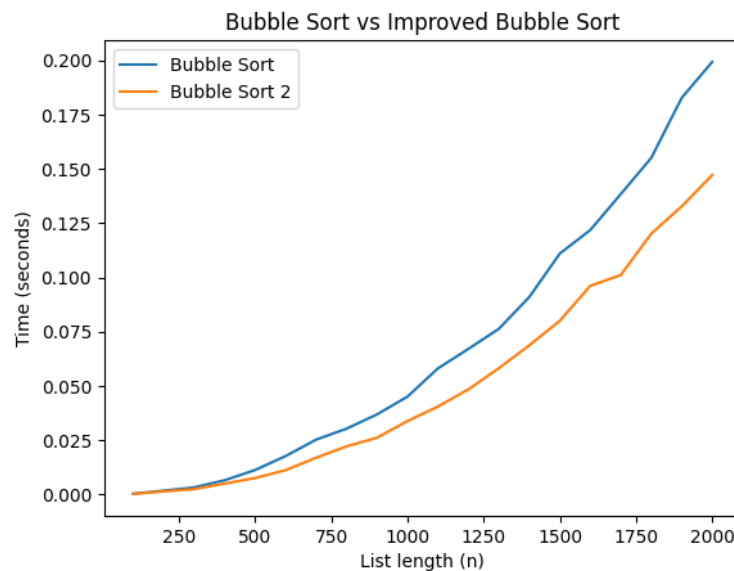


Figure 2: performance of bubble sort vs improved bubble sort

Moving on, our selection sort also displays an improvement. We essentially get double the work done for every outer iteration, by fetching both the minimum and maximum index,

since we know we have to iterate the entire list. Still, Figure 3 does not show a 2 times speed up as might be expected though. This is because while we are now efficient in our outer loop, the inner loop has double the comparisons to work through now that we have to make two checks per element, for both the min and max. This checks out, and we only see a marginal overall improvement for our improved selection sort.

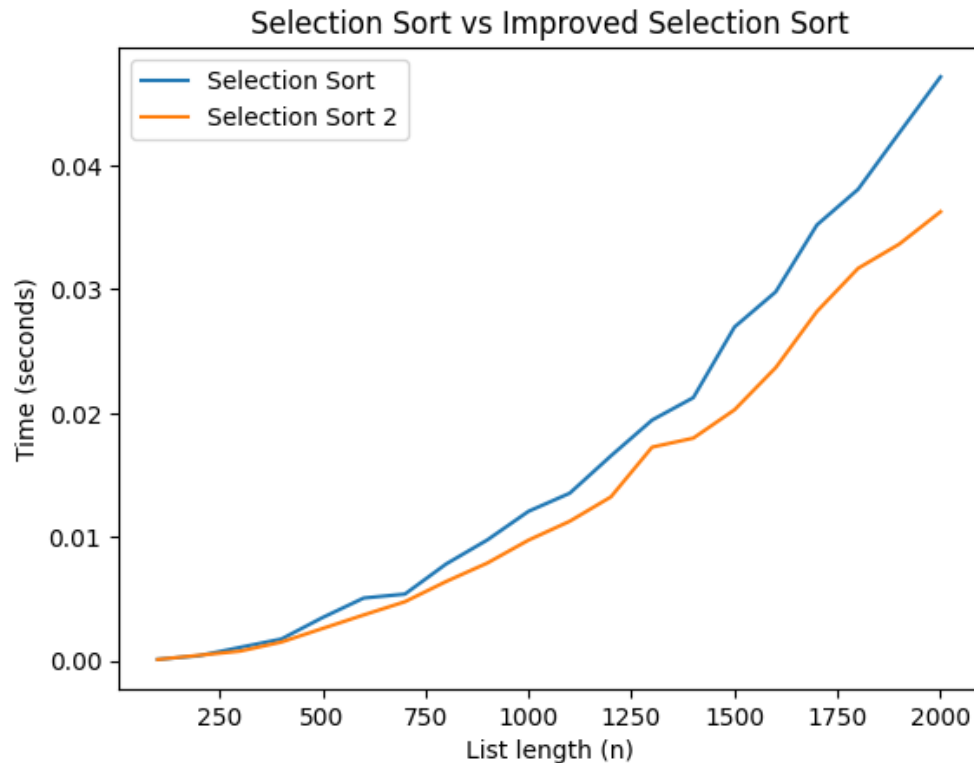


Figure 3: performance of selection sort vs improved selection sort

3 Experiment 3: Bad Algorithms on Special Lists

3.1 Objective

Our goal is to investigate the effect of running the three bad algorithms on lists with special properties. Specifically, we will be testing on lists that have had a random number of swaps. In other words, for a fixed size list, we want to see how these three algorithms react to a list with almost no swaps (almost sorted) up to a list with many swaps, which becomes an almost completely random array (specifically, at $\text{swaps} = \text{length} * \log(\text{length}) / 2$)

3.2 Methodology

First, we set out list length to test to 2000, which gives a hefty enough size to notice some differences, but not too large for quadratic algorithms to handle. We tested 10 numbers of swaps, from 0 to 3301 (which results in an indistinguishably random list for 2000 elements, based on the provided algorithm). for each number of swaps on the list of length 2000, we run 5 tests to eliminate noise.

3.3 Results

The takeaways are quite clear; Figure 4 shows selection sort is the most consistent out of the three. It has no real advantage on a nearly sorted list, but also performs the best on lists that are completely random. All in all, it remains at a constant run time throughout, which makes sense given the number of comparisons and swaps are not affected by the order of the initial array.

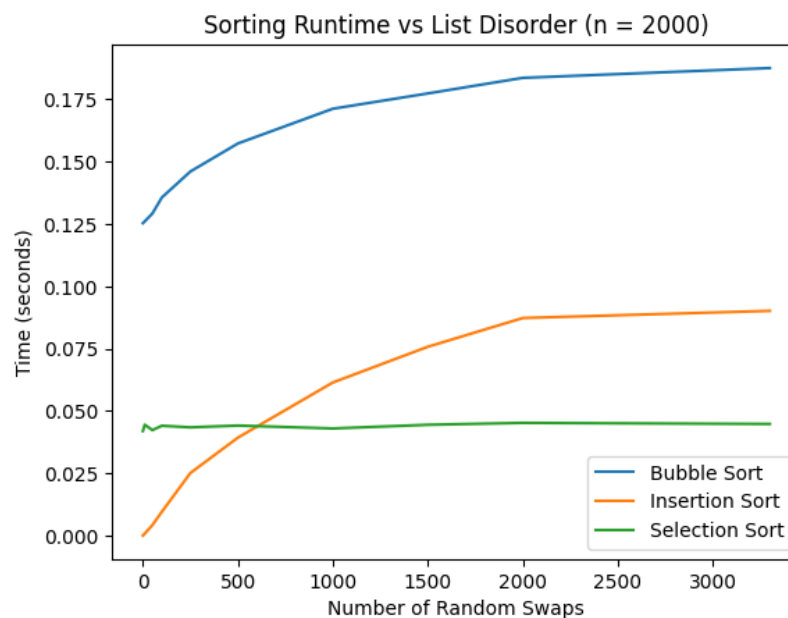


Figure 4: run time of bad sorts on lists with 0 swaps, up to 3301 swaps

Insertion sort is an interesting case. For a fully or nearly sorted list, insertion sort is almost instant, since we only perform $n - 1$ comparisons, then no swaps. This makes it extremely fast, especially compared to the rest of the sorts which still require quadratic time. As the disorder of the list increases, insertion sort slowly falls behind selection sort, since it requires more total comparisons and swaps when the list has been shuffled.

Finally, bubble sort is the worst sort by all metrics. It does perform slightly better on a sorted list and becomes less inefficient on a disordered list, however the redundant swaps even on a sorted list make its run time already terrible, even when the list is almost sorted. This would be a great way to see the efficiency of our improved bubble sort (see chapter 2), since we would avoid a lot of the redundancy and have a much lower initial run time. Bubble sort follows the same trajectory as insertion sort, however it worse than both other sorts at all points.

4 Experiment 4: Run Times of Good Algorithms

4.1 Objective

Our goal is to investigate the run time of the three good sorts. While they have similar overall run times, the differences in behavior can lead to a drastic difference in performance.

4.2 Methodology

Since these algorithms are much more efficient than our previous bad sorts, we can test with much larger lists. We tested on about 200 lists from length 100 to 20001, running 5 tests for each length and taking the best performance to reduce external influence. Since we have more optimal sorts, we check a wider range of lists at a much larger size. Otherwise, our testing is quite standard; we generate random lists in the range and plot the results.

4.3 Results

From our results in Figure 5, quick sort is by far the fastest. Merge sort performs slightly worse, though it still outperforms heap sort which is by far the worst performing out of the better sorts. The trend is also quite consistent, meaning this holds both for smaller lists, as well as extremely large lists.

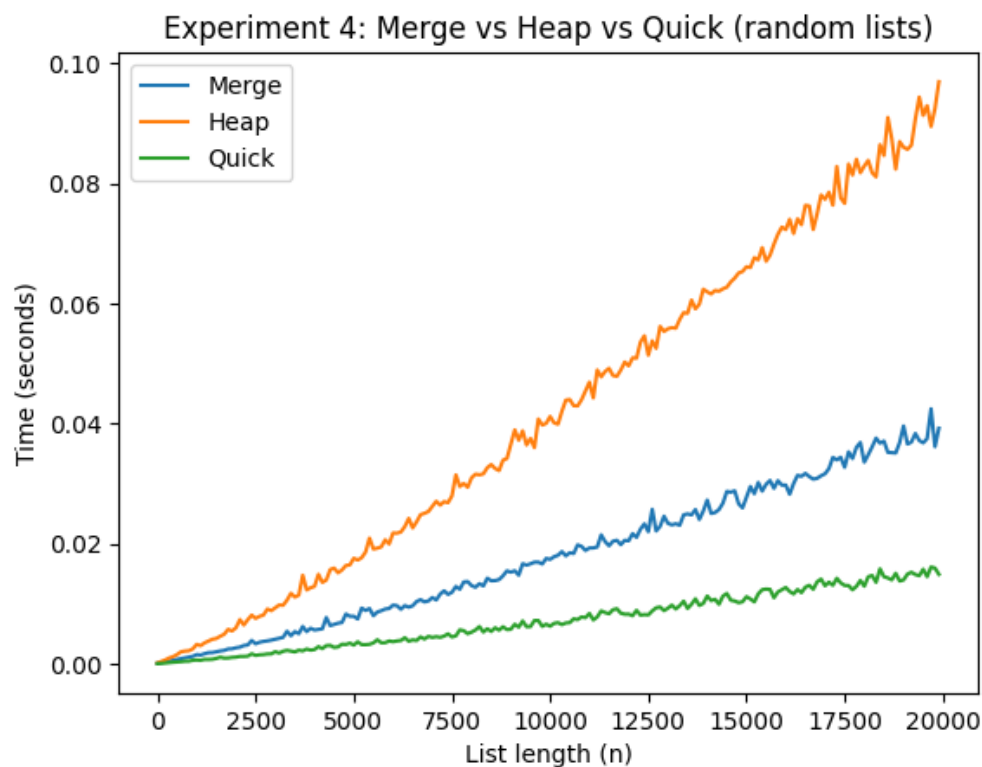


Figure 5: run time of the three good sorts

5 Experiment 5: Optimal Conditions for Quick Sort

5.1 Objective

The performance of quick sort is extremely volatile, as it is heavily dependent on the chosen pivot. Since we cannot efficiently choose a pivot that splits the list into better than 75/25 while also maintaining $O(n \log n)$ complexity, we end up with a worst case scenario where we might choose the largest or smallest value in the list as a pivot, and we end up having blocks that are almost the size of the whole list. If a list is sorted in reverse order, our quick sort does the worst, since we always pick the first element as the pivot. We will experiment to test how many swaps are required to go from the worst case scenario of quick sort, to a variation that is the fastest sort on average.

5.2 Methodology

One important note: We had to manually increase the recursion depth using python's `sys.setrecursionlimit(200000)`

since our quick sort degrades so bad that we end up with errors due to exceeding the maximum default recursion depth (since we always recurse on the entire remaining list). We set a few values to check with swaps, from 0 to 2000. These are increasing intervals, so we checked from [0, 1, 5, 10, 25, 50, 100, 250, 500, 1000, 2000]. This gives us a nice variety and a wide enough range. We fix the length of the list to be 5000, large enough to see a difference between the three. For each swap value, we run 5 times to eliminate noise, as well as generating multiple types of swapped lists and averaging the results to have a wide distribution of disorder.

5.3 Results

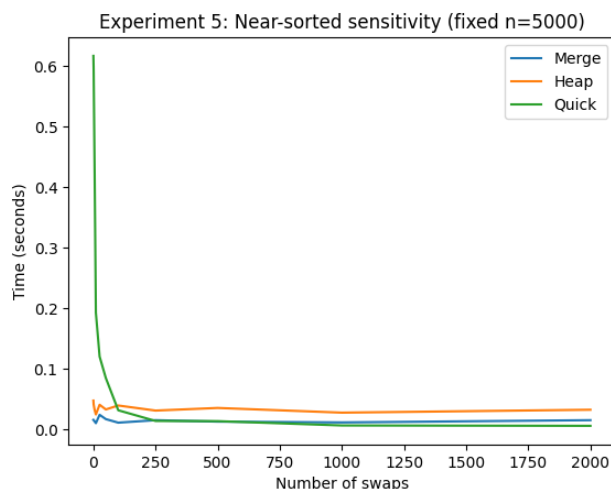


Figure 6: performance of three good sorts on reversed lists with x swaps

We see exactly why quick sort is $O(n^2)$ in the worst case here: Figure 6 shows that quick sort is by far the worst sort until around the 100 swap mark, after which choosing the first element is likely not going to cause the worst case scenario of $n - 1$ elements on the split. Otherwise, the behaviour of the sorts is quite standard. Both heap sort and merge sort perform essentially identically no matter the number of swap. Merge sort performs better than heap sort all throughout, and better than quick sort until around 250 swaps. Then, quick sort continues to improve until it is consistently much faster than both the other sorts. Essentially, if the list is random enough, quick sort will perform better.

6 Experiment 6: Better Quick Sort

6.1 Objective

As we learned previously, quick sort is extremely fast but has a few vulnerabilities. We will see if it is possible to improve on our previous algorithm by using 2 pivots, instead of just 1, where we will now have two fixed points and 3 sections.

6.2 Methodology

For the most part, this new algorithm is built on the previous implementation, just improved to work with three pivots. We handle base cases of length 0, 1, or 2 first. Then we choose the two pivots as the first and last values of the array (which could cause issues, but is the compromise we make). From there, we just connect the sections based on the pivot values.

Listing 3: Quick Sort with 2 Pivots (creates a copy)

```
def dual_quicksort_copy(L):
    n = len(L)
    if n <= 1:
        return L[:]
    if n == 2:
        return L[:] if L[0] <= L[1] else [L[1], L[0]]
    p = L[0]
    q = L[-1]
    if p > q:
        p, q = q, p
    left = []
    middle = []
    right = []
    for x in L[1:-1]:
        if x < p:
            left.append(x)
        elif x > q:
            right.append(x)
        else:
            middle.append(x)
    return dual_quicksort_copy(left) + [p] +
           dual_quicksort_copy(middle) +
           [q] + dual_quicksort_copy(right)
```

We had an issue when testing which was to be expected; we ended up passing the default recursion limit in python! This could be circumvented via an iterative implementation, however we decided to repeat what we did in experiment 5 and manually increased the recursion limit. We tested 8 lists of lengths from 10 to 20000, testing 5 runs on each length. This was sufficient data to get a picture of the comparison between the two sorts.

6.3 Results

Figure 7 displays the difference in performance of standard (single-pivot) quick sort against dual-pivot quick sort. The results demonstrate that dual-pivot quick sort is faster across all tested lengths ($N=10$ to $20,000$). The divergence of the two curves highlights that dual-pivot quick sort achieves a lower constant factor in its time complexity. They have the same overall complexity, both still being $O(n \log n)$, yet 2 pivots consistently out performs as the size of the list grows and continues to hold. The first reason is that mathematically, more pivots leads to slightly less overall comparisons. However, this alone does not explain the massive difference. Another reason for this efficiency is how modern computers work. A 3-way split is likely handled much more efficiently by modern CPU caches as they have the ability to move around memory better, and benefit from this flexibility.”

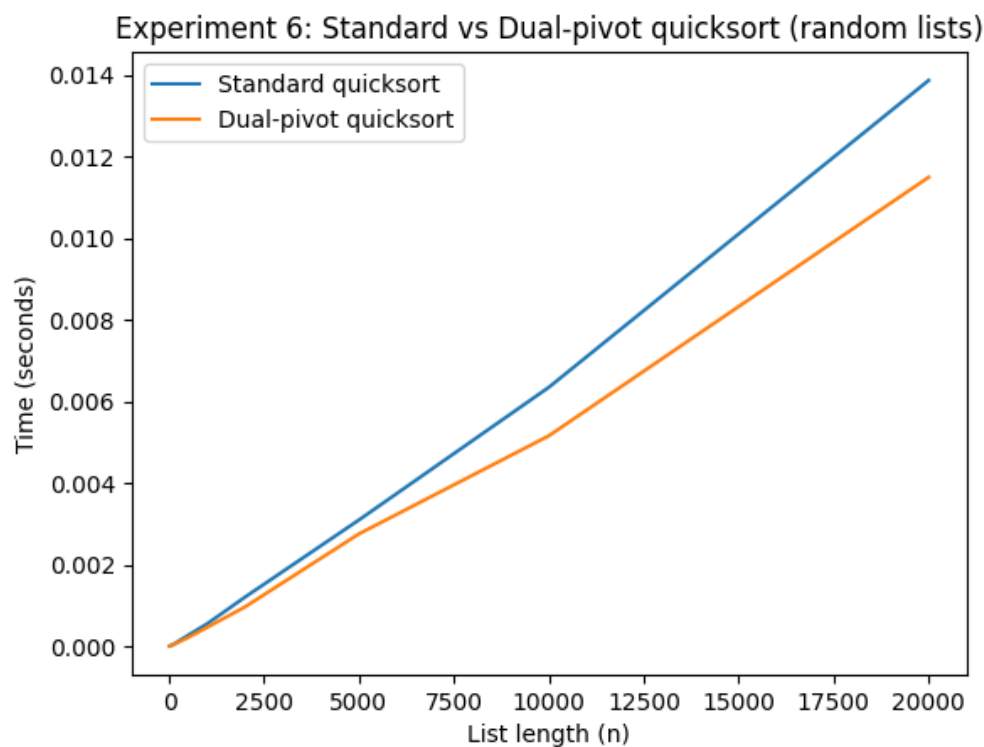


Figure 7: performance of quick sort with 1 pivot vs 2 pivots

7 Experiment 7: Iterative Merge Sort

7.1 Objective

Recursive merge sort is very efficient as a sorting algorithm, but iteration almost always beats recursion in performance. We will try to build an iterative version of merge sort, then compare the results with our original recursive merge sort. We will draw conclusions regarding the differences between the two implementations and the implications of their performance.

7.2 Methodology

In recursive merge sort, we break down the list into the smallest blocks, then join them together in the right order. Working bottom-up, instead of breaking down the list into chunks then combining them, we create a 'sliding window' for each section we merge. We have an initial width of a block that starts at 1, combining two adjacent elements at a time. We do this for all possible pairs, each with size = width * 2, then we double the size of our width and continue. Python slicing comes in handy, as we can write a clean implementation that handles edge cases due to slicing only giving what could be there, and our merge function is fine with mismatched or empty lists.

Listing 4: Iterative Merge Sort Logic

```
def bottom_up_mergesort(L):
    width = 1
    while width < len(L):
        for i in range(0, len(L), 2 * width):
            left = L[i : i + width]
            right = L[i + width : i + 2 * width]
            L[i : i + 2 * width] = merge(left, right)
        width *= 2
```

All the logic is implemented in two nested loops, where the inner loop iterates over all pairs in the list (worst case $O(N)$) and the outer loop doubles the width until we reach a width that encompasses the whole array (worst case $O(\log n)$). The algorithm runs in $O(n \log n)$ time, as expected of a merge sort, and the same as the previous recursive merge sort. We use the same implementation of 'merge' used in the recursive version, just re-purposing to work with our sliding window. After the implementation, we will test both on around 50 lists of lengths between 100 and 50000, running about 20 trials per list to ensure we do not have interruption from background noise.

7.3 Results

As we can see in Figure 8, the iterative implementation outperforms the recursive in all lists of relevant length.

Runtime of recursive mergesort vs iterative mergesort for random lists of length 100 to 5,000

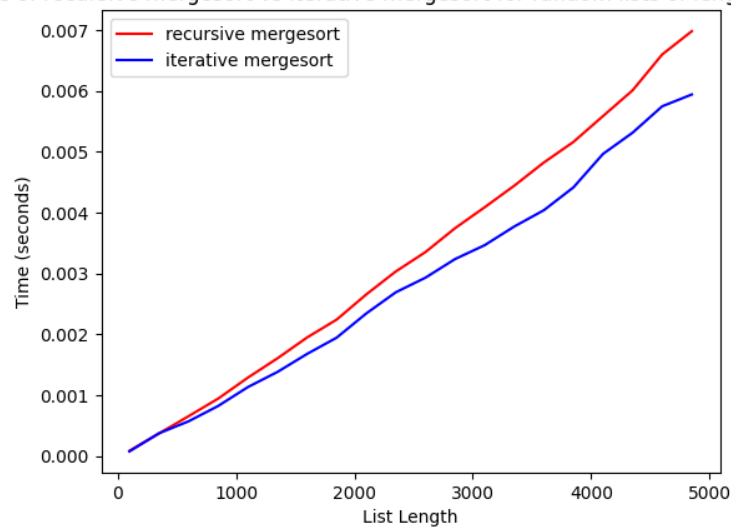


Figure 8: iterative merge sort outperforms recursive for large lists

The trend for both is $O(n \log n)$, but in the new implementation we have are free of the expensive stack frames created by recursion. It may be surprising to see the difference in performance is not that drastic, as it was when we wrote recursive vs iterative binary search. This makes sense though, since the difference between iterative and recursive implementation is just the $O(\log n)$ operations of separating blocks to merge, not the $O(n)$ merging of blocks, which is the same between both and cannot really be improved by using iteration (it does not use recursion in the first place). We see a clear constant improvement, but as we cannot improve the 'merge' part of the merge sort, our optimization is limited.

8 Experiment 8: Insertion Sort vs 'Good' Sorts

8.1 Objective

Insertion sort with its $O(n^2)$ complexity is clearly less efficient than merge sort and quick sort for large lists. However, for smaller lists, there are cases where a well-written insertion sort can consistently outperform the two sorts, and there are cases where insertion sort will never out perform the other two. We will run tests on unique lists

8.2 Methodology

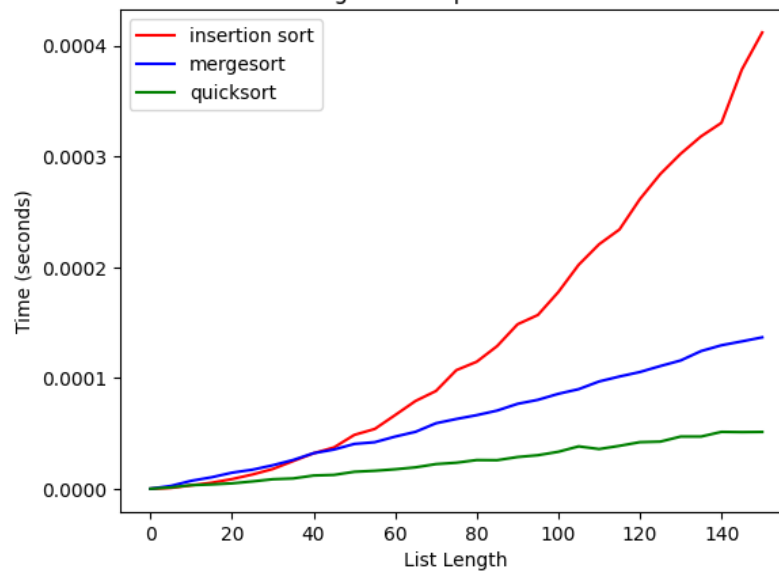
First, we must preface all observations regarding insertion sort with the fact that it has a time complexity of $O(N^2)$. Our first tests were on completely random lists up to length 150, to test the lengths of lists at which insertion becomes unfeasible. Then, using this, we ran tests on two types of lists: nearly sorted lists, and reverse sorted lists (i.e., furthest from sorted). Insertion sort is 'adaptable', meaning it can work better on smaller lists where certain conditions are fulfilled that allow it to skip a lot of work the merge sort and quick sort would still need to do. We test insertion sort's best case on lists of length 50; too little elements, then insertion sort would basically always perform better no matter the circumstances; too many elements, then insertion sort would be destroyed by the asymptotic nature no matter the condition, and our data would be less dense while not providing valuable information.

When testing with nearly sorted lists, we test on 0 - 150 swaps, going from lists with very little change to maximized change (since swapping has diminishing return, 150 swaps shakes up a list of 60 elements enough in testing). For each list, the number of inversions will be counted, where an 'inversion' is defined as a pair of indices (i, j) such that $i < j$ but $L[i] > L[j]$. This will be used to compare the number of inversions to the performance for each sort. Finally, reverse sorted lists are quite simple; we check performance on lists of lengths up to 60 with this feature. Note that for each test, at minimum a few hundred tests are run for each parameter, ensuring there is little outside interference

8.3 Results

First, we can see in Figure 9, insertion sort is simply not feasible on lists larger than 100 elements. As we will see later on, even in the best case it does not make sense to use insertion sort vs the other sorts at these list sizes. To see where insertion sort shines, we need to understand how insertion sort works. In short, insertion sort fixes all 'inversions' in the list. Here, we find a clue to what makes insertion sort useful. In short, insertion sort iterates once over all elements in the list, and fixes each inversion. We can model the run time as $O(n + I)$, where n is the length of the list and I is the number of inversions. Figure 10 exemplifies this; the performance of insertion sort is directly proportional to the number of inversions. What this means is that for a list of length 60, when the number of inversions is around 200, insertion sort beats the other best sorts! It even continues to be better than merge sort until the number of inversions in the list crosses 500.

Runtime of insertion sort vs mergesort vs quicksort for random lists of length 0 to 150

Figure 9: insertion sort can only compete up to lists of length ≈ 50

The closer a list is to sorted, the less inversions there are in total, meaning insertion sort works very efficiently. It also gives insight into the performance of insertion sort in average and worst cases. First, we can infer from this information the worst case scenario for insertion sort is when we have to perform maximum inversions, and by our definition of inversion, that would mean for every index, the element is larger than all indices that come after it, in other words, a reverse sorted list! By testing we can indeed confirm insertion sort is worst with these lists, as its run time blows past both sorts.

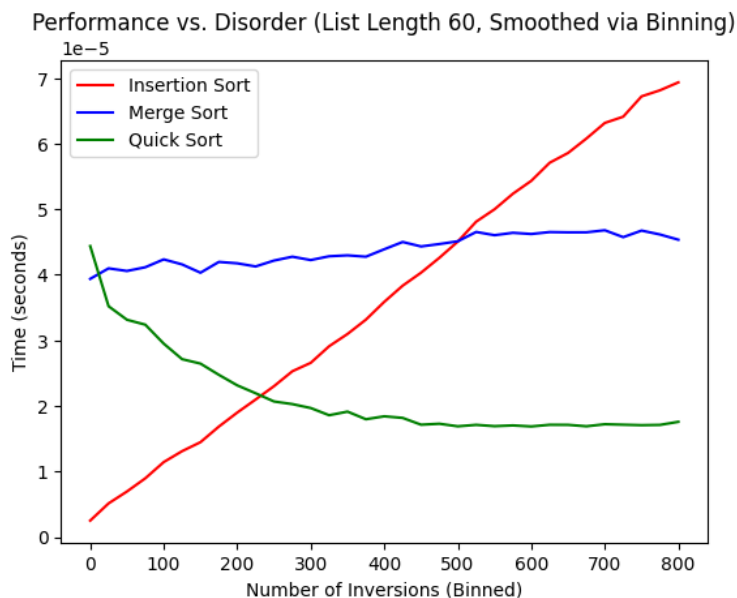


Figure 10: insertion sort performance correlates directly with the number of inversions

We can do some math to understand how this helps us. In the worst case scenario we described, the maximum number of inversions is easy to calculate, since there are n inversions for the first index, $n - 1$ for the next, etc. which sums to $\frac{n(n-1)}{2}$. Using this worst case, we ran tests on the ratio of how many inversions we had compared to the maximum possible. After running hundreds of tests, we found that the average inversion ratio was 0.5021, that is, $\approx 50\%$ of $\frac{n(n-1)}{2}$ possible inversions, which means on average the number of inversions in a list is $\approx \frac{n^2}{4}$, and the run time is $O(n + \frac{n^2}{4})$. As a final conclusion, we can note that with our data, we could combine a very efficient sorting algorithm like merge sort with a specialized sorting algorithm like insertion sort, which would be efficient on very short lists. Since merge sort breaks down the list into such small pieces, we could sort smaller chunks with insertion sort, instead of having to run merge on all smaller chunks. When we find a threshold such that insertion sort is more effective on that size of chunks than running merge sort further, we could have a merge sort that is noticeably more efficient. In fact, we already have a rough idea of what size of list we should do this, it should be on sections of size around 20, or chunks of list with less than 500 inversions.

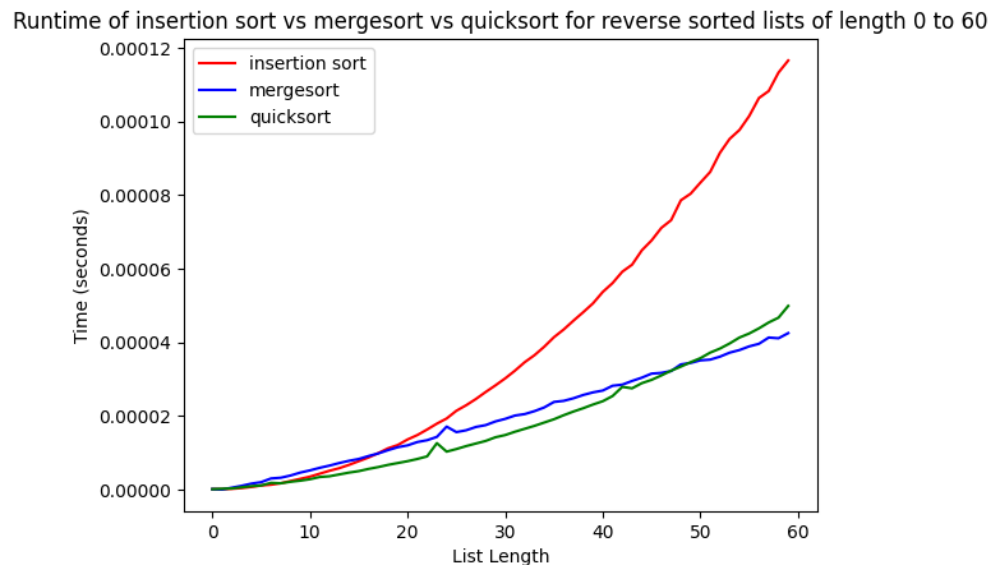


Figure 11: insertion sort struggles with reverse sorted lists

A Code Navigation Guide

To verify the results presented in this report, please refer to the attached Jupyter Notebook files. All code was run on a Python 3.14+ kernel. **!!BEFORE RUNNING!!** You must have matplotlib installed on the kernel before running any of the notebooks. This can be done by running 'pip install matplotlib' in the environment. Here is a guide on what is attached:

- **experiments/auxiliary/**: Contains both bad_sorts.py and good_sorts.py, used in experiments 1 - 8
- **experiments/**: Contains experiments 1 - 8. To run, just run all cells starting from the top. Any extra information will be contained in the respective notebooks
- **img/**: Contains all relevant graphs from each experiment, grouped with the respective experiment